

```

/* 1. Prime Number Check */
#include <stdio.h>
#include <math.h>

int isPrime(int n) {
    if (n <= 1) return 0;
    for (int i = 2; i <= sqrt(n); i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}

int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (isPrime(num))
        printf("%d is a prime number\n", num);
    else
        printf("%d is not a prime number\n", num);
    return 0;
}

```

```

/* 2. Armstrong Number Check */
#include <stdio.h>
#include <math.h>

int main() {
    int num, originalNum, remainder, result = 0, n = 0;
    printf("Enter a number: ");
    scanf("%d", &num);
    originalNum = num;

    // Count number of digits
    for (originalNum = num; originalNum != 0; n++) {
        originalNum /= 10;
    }

    originalNum = num;

    // Calculate result

```

```

while (originalNum != 0) {
    remainder = originalNum % 10;
    result += pow(remainder, n);
    originalNum /= 10;
}

if (result == num)
    printf("%d is an Armstrong number\n", num);
else
    printf("%d is not an Armstrong number\n", num);
return 0;
}

```

/ 3. Perfect Number Check */*

```
#include <stdio.h>
```

```

int main() {
    int num, sum = 0;
    printf("Enter a number: ");
    scanf("%d", &num);

    for (int i = 1; i < num; i++) {
        if (num % i == 0) {
            sum += i;
        }
    }

    if (sum == num)
        printf("%d is a Perfect number\n", num);
    else
        printf("%d is not a Perfect number\n", num);
    return 0;
}

```

/ 4. Strong Number Check */*

```
#include <stdio.h>
```

```

int factorial(int n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}

```

```

}

int main() {
    int num, originalNum, sum = 0, digit;
    printf("Enter a number: ");
    scanf("%d", &num);
    originalNum = num;

    while (num > 0) {
        digit = num % 10;
        sum += factorial(digit);
        num /= 10;
    }

    if (sum == originalNum)
        printf("%d is a Strong number\n", originalNum);
    else
        printf("%d is not a Strong number\n", originalNum);
    return 0;
}

```

/ 5. 1D Array Operations */*

```

#include <stdio.h>

int main() {
    int arr[100], n;
    printf("Enter the size of array: ");
    scanf("%d", &n);

    // Input array elements
    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Display array
    printf("\nArray elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}

```

```

// Find sum
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
printf("\nSum of elements: %d", sum);

// Find max and min
int max = arr[0], min = arr[0];
for (int i = 1; i < n; i++) {
    if (arr[i] > max) max = arr[i];
    if (arr[i] < min) min = arr[i];
}
printf("\nMaximum: %d\nMinimum: %d\n", max, min);
return 0;
}

```

```

/* 6. Basic 2D Matrix Operations */
#include <stdio.h>

int main() {
    int mat1[10][10], mat2[10][10], result[10][10];
    int rows, cols;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Input first matrix
    printf("Enter elements of first matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &mat1[i][j]);
        }
    }

    // Input second matrix
    printf("Enter elements of second matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &mat2[i][j]);
        }
    }
}

```

```

    }

    // Add matrices
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result[i][j] = mat1[i][j] + mat2[i][j];
        }
    }

    // Display result
    printf("\nResultant Matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", result[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

/* 7. Matrix Diagonal Operations */

#include <stdio.h>

```

int main() {
    int matrix[10][10], n;
    int leftDiagonalSum = 0, rightDiagonalSum = 0;

    printf("Enter size of square matrix: ");
    scanf("%d", &n);

    printf("Enter matrix elements:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Calculate diagonal sums
    for(int i = 0; i < n; i++) {
        leftDiagonalSum += matrix[i][i]; // Left diagonal
        rightDiagonalSum += matrix[i][n-1-i]; // Right diagonal
    }
}

```

```

    printf("Left diagonal sum: %d\n", leftDiagonalSum);
    printf("Right diagonal sum: %d\n", rightDiagonalSum);
    return 0;
}

```

```

/* 8. Upper and Lower Triangle Sum */
#include <stdio.h>

```

```

int main() {
    int matrix[10][10], n;
    int upperSum = 0, lowerSum = 0;

    printf("Enter size of square matrix: ");
    scanf("%d", &n);

    printf("Enter matrix elements:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Calculate sums
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(j > i) upperSum += matrix[i][j]; // Upper triangle
            if(j < i) lowerSum += matrix[i][j]; // Lower triangle
        }
    }

    printf("Upper triangle sum: %d\n", upperSum);
    printf("Lower triangle sum: %d\n", lowerSum);
    return 0;
}

```

```

/* 9. Identity Matrix Checker */
#include <stdio.h>

```

```

int main() {
    int matrix[10][10], n;

```

```

int isIdentity = 1;

printf("Enter size of square matrix: ");
scanf("%d", &n);

printf("Enter matrix elements:\n");
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        scanf("%d", &matrix[i][j]);
    }
}

// Check if identity matrix
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        if((i == j && matrix[i][j] != 1) || (i != j && matrix[i][j] != 0)) {
            isIdentity = 0;
            break;
        }
    }
}

if(isIdentity)
    printf("The matrix is an identity matrix\n");
else
    printf("The matrix is not an identity matrix\n");
return 0;
}

```

/* 10. Array Reversal */

#include <stdio.h>

```

int main() {
    int arr[100], n;

    printf("Enter size of array: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(int i = 0; i < n; i++) {

```

```

        scanf("%d", &arr[i]);
    }

    printf("Original array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    // Reverse array
    for(int i = 0; i < n/2; i++) {
        int temp = arr[i];
        arr[i] = arr[n-1-i];
        arr[n-1-i] = temp;
    }

    printf("\nReversed array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}

/* 11. Matrix Transpose */
#include <stdio.h>

int main() {
    int matrix[10][10], transpose[10][10], rows, cols;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    printf("Enter matrix elements:\n");
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Calculate transpose
    for(int i = 0; i < rows; i++) {

```

```

        for(int j = 0; j < cols; j++) {
            transpose[j][i] = matrix[i][j];
        }
    }

    printf("\nTransposed matrix:\n");
    for(int i = 0; i < cols; i++) {
        for(int j = 0; j < rows; j++) {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

/* 12. Row and Column-wise Operations */
#include <stdio.h>

```

int main() {
    int matrix[10][10], rows, cols;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    printf("Enter matrix elements:\n");
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Row-wise operations
    printf("\nRow-wise sums and averages:\n");
    for(int i = 0; i < rows; i++) {
        int rowSum = 0;
        for(int j = 0; j < cols; j++) {
            rowSum += matrix[i][j];
        }
        float rowAvg = (float)rowSum / cols;
        printf("Row %d - Sum: %d, Average: %.2f\n", i+1, rowSum, rowAvg);
    }
}

```

```
}

// Column-wise operations
printf("\nColumn-wise sums and averages:\n");
for(int j = 0; j < cols; j++) {
    int colSum = 0;
    for(int i = 0; i < rows; i++) {
        colSum += matrix[i][j];
    }
    float colAvg = (float)colSum / rows;
    printf("Column %d - Sum: %d, Average: %.2f\n", j+1, colSum, colAvg);
}
return 0;
}
```